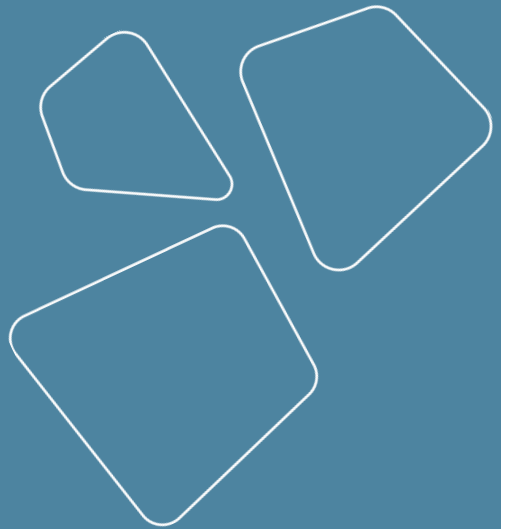
Code management and git servers

Vladislav Goncharenko

MIPT, spring 2025

Recap

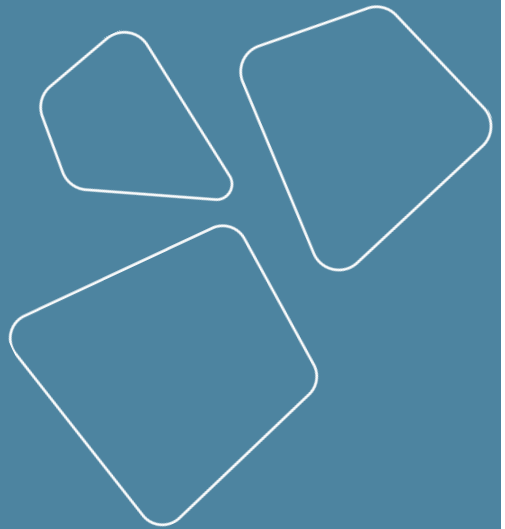


- Зачем нужен MLOps
- Стандарты разработки Data Science проектов
- Что такое MLOps
- Подзадачи MLOps
- Обзор решений от крупных компаний
- Проблемы open source стека

Practice:

- Пакетирование в Питоне
- Управление зависимостями
  - poetry
- Jupyter server с паролем

Outline



- Code management
- Git servers
- Workflow
  - Distribution
- CI/CD
- Codebase structure
  - Code formatting
  - Inline documentation
- pre-commit
- code quality tools

Code management

girafe
ai

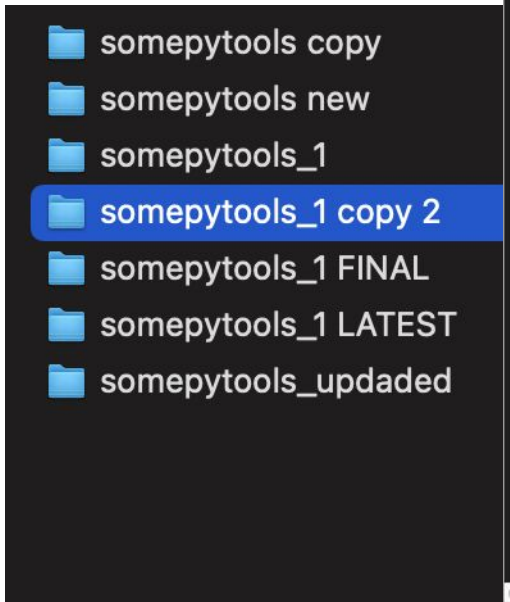
01

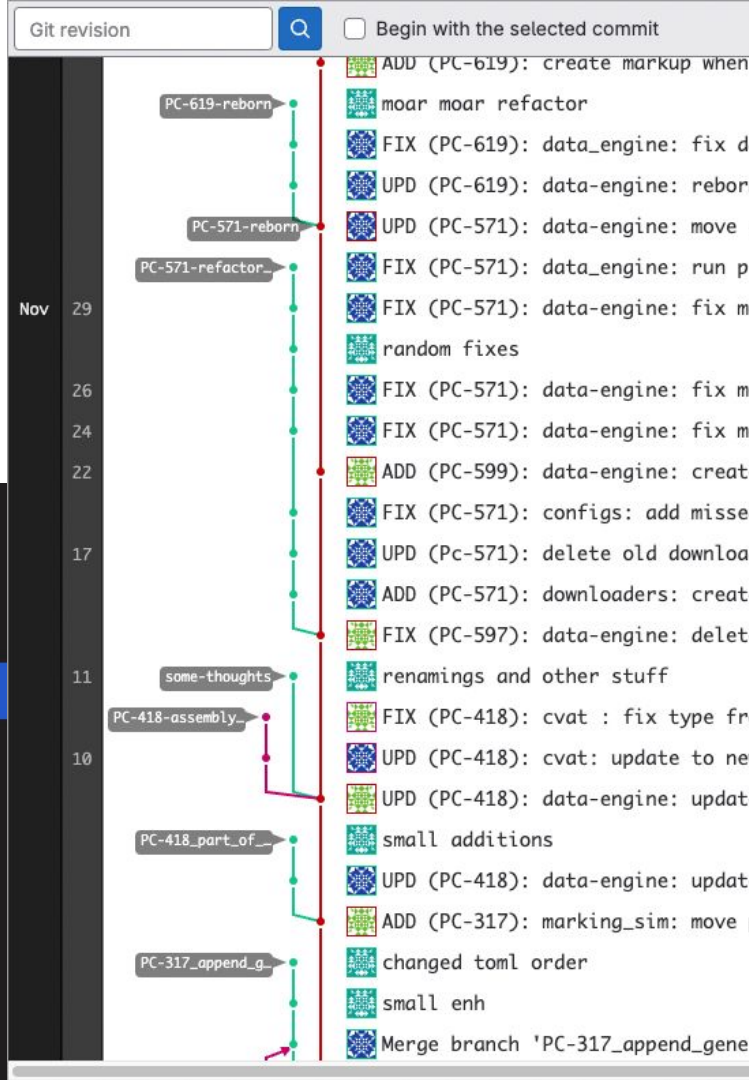
Code management

- One local copy
- Several local copies

Code management

- One local copy
- Several local copies
- Remote copies
- Version control system (VCS)



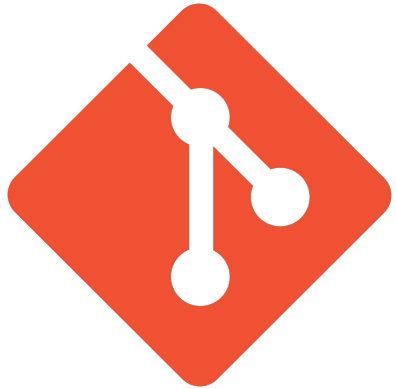


Version control systems (VCS)

- Local only
  - Source Code Control System (SCCS) – part of UNIX [1973]
- Client–server
  - Apache Subversion (SVN) [2000]
  - Yandex Arcadia (fork of SVN)
  - master branch is called trunk

Version control systems (VCS)

- Local only
  - Source Code Control System (SCCS) [1973]
- Client–server
  - Apache Subversion (SVN) [2000]
  - Yandex Arcadia (fork of SVN)
- Distributed
  - Git [2005]
  - Mercurial [2005]
  - Plastic (.NET) [2006]
  - Perforce [1995]



PERFORCE

mercurial

Learn git (if you need)

Brilliant git tutorial by Atlassian:

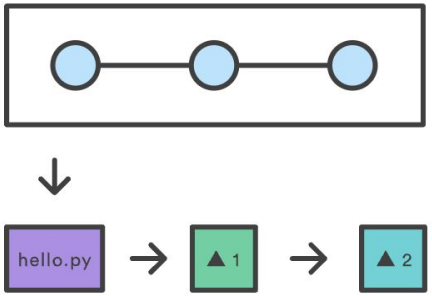
https://www.atlassian.com/git/tutorials/what-is-version-control

Covers topics from basics to advanced.
Can be read cover to cover.

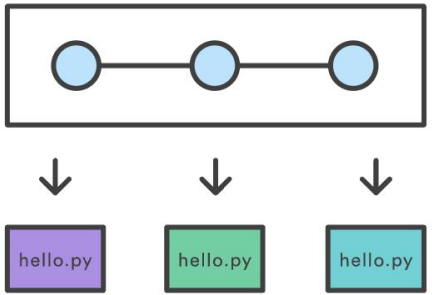
Interactive tutorial https://learngitbranching.js.org/

Complete git reference https://git-scm.com/book/en/v2

Recording File Diffs (SVN)



Recording Snapshots (Git)



Git servers

- GitHub [2008]
  - proprietary
  - owned by Microsoft
- Atlassian Bitbucket [2008]
- GitLab [2011]
  - open source, but not free
- Gogs [2014]
  - FOSS
  - example of running service - GIN
- Gitea [2016]
  - fork of Gogs FOSS

Other options: https://alternativeto.net/software/github/?license=opensource

e.g. https://codeberg.org/

Badges in README.md on github

What to store in repo

- Use gitignore - it is mandatory
  - global: exclude OS dependent files
    - e.g. .DS_store for macOS
  - local: exclude project and language specific files
    - e.g. pycache, data files
- Code
- Configs

Not to store:

- Data
- Model artifacts
- Visualizations

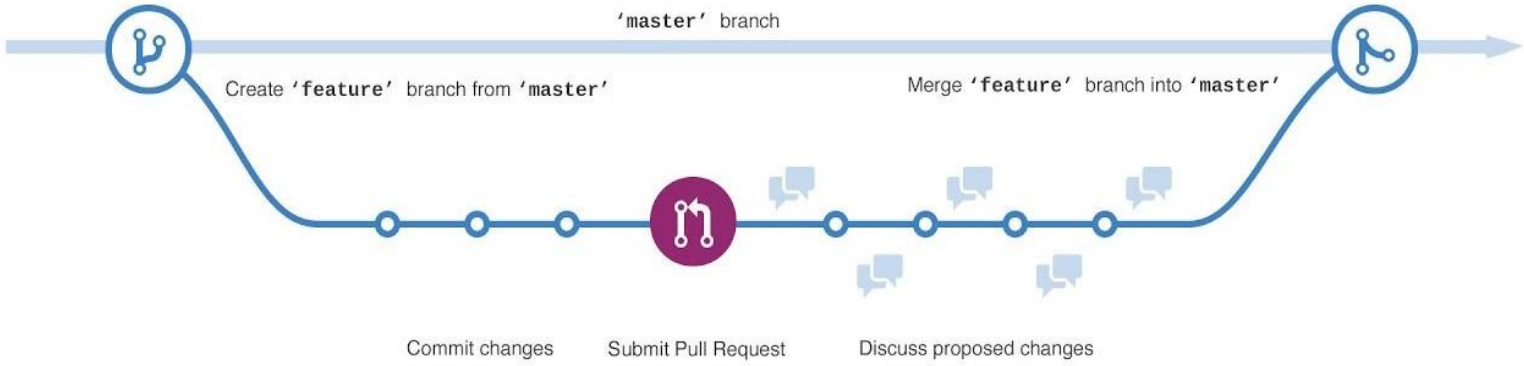
Git workflow

girafe
ai

02

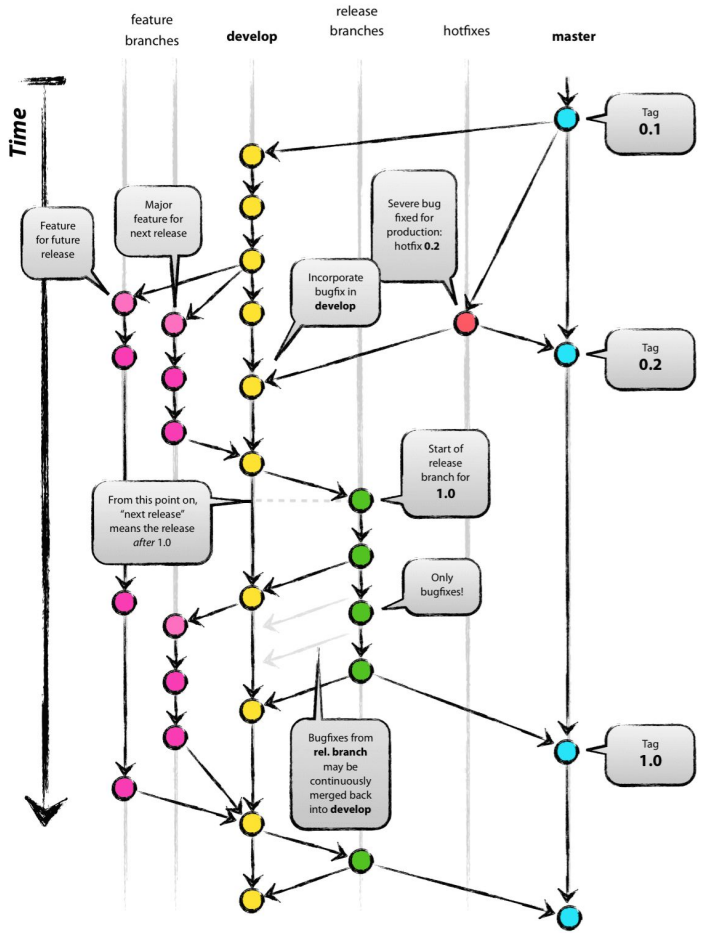
Teamwork

- Best practice - merge requests (pull requests)

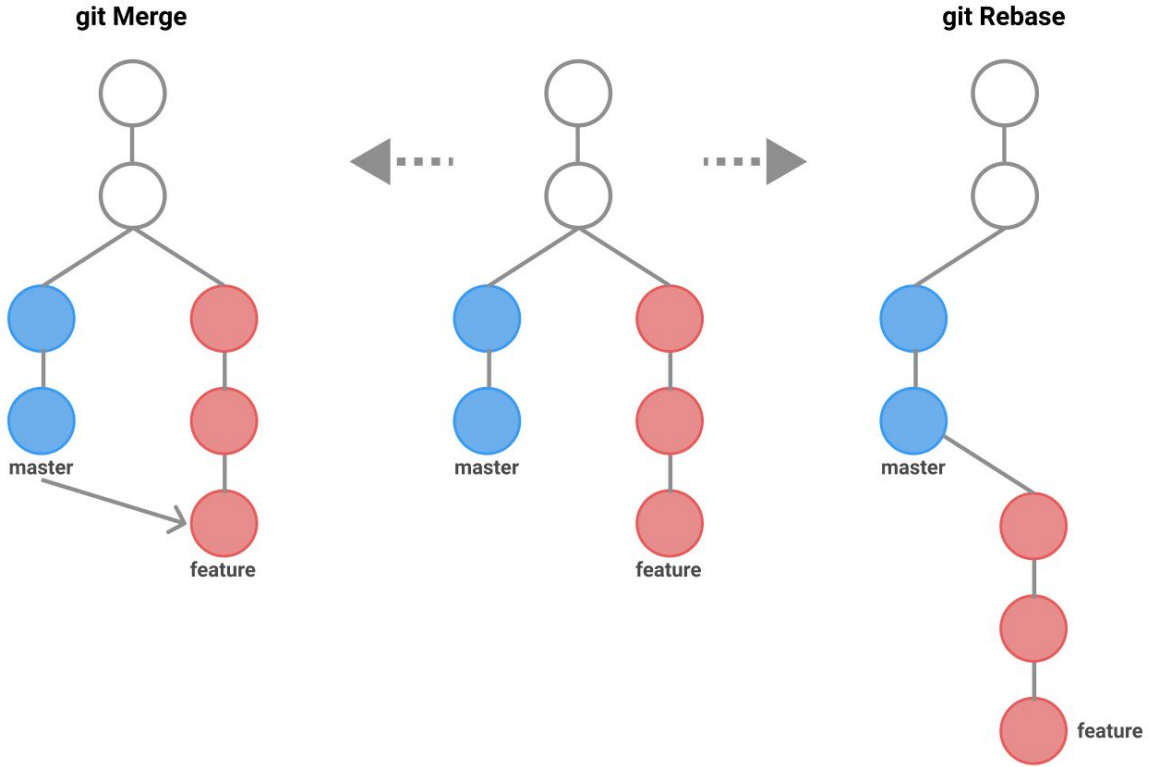


Teamwork

- Best practice - merge requests (pull requests)
  - Advanced - Git Flow (original article) [2010]
  - Flows comparison



Git rebase


git Merge
master
feature
master
feature
git Rebase
master
feature

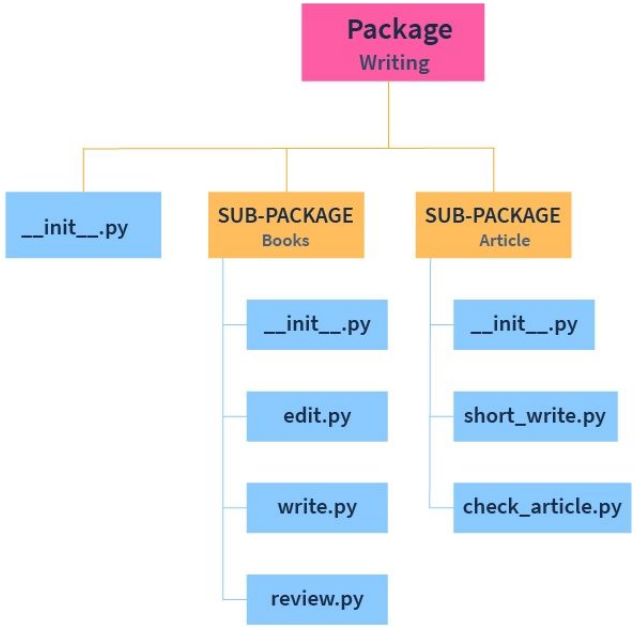
Git client implementations

- on JavaScript https://isomorphic-git.org/
-

Git parent search algorithm

Distribution

- Source code
  - clone git repo to final machine
  - update with git pull
  - doesn't need structure
- Packages
  - install with pip install
  - update with pip update
  - requires structure
  - for simple packaging use poetry
- Special tools
  - ONNX
  - TensorRT
- Special services
  - Triton inference server
  - MLflow serving



Continuous Integration (CI)
&
Continuous Delivery (CD)

girafe
ai

03

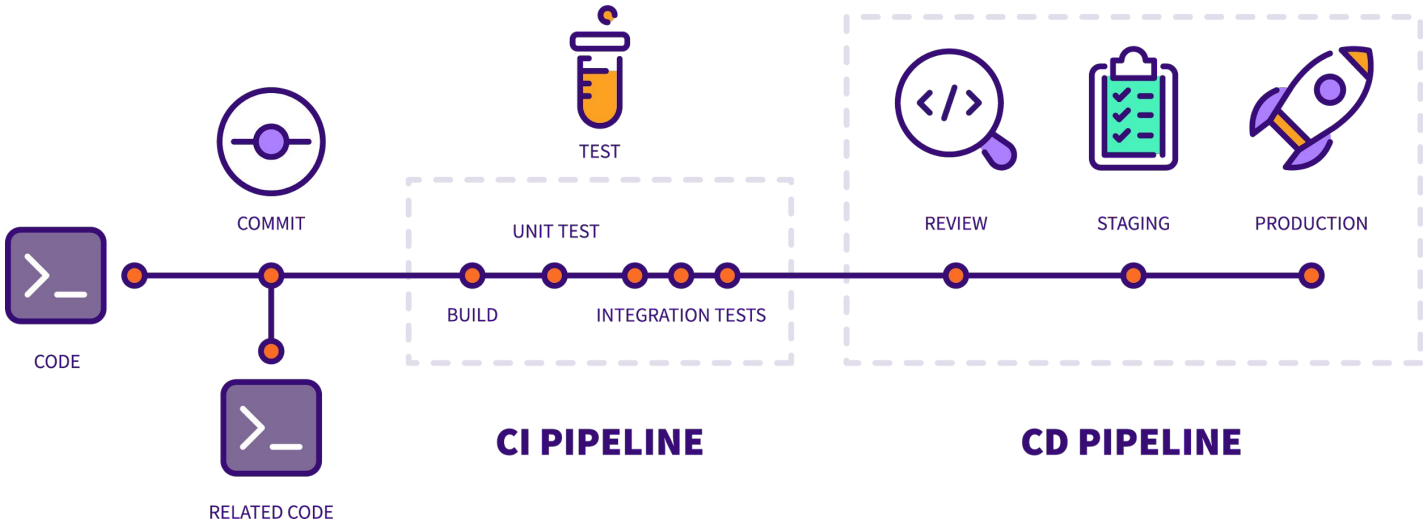
CI/CD stacks

Git server integrated

- Github Actions
- GitLab CI
- Gitea Actions
- Atlassian Bamboo

Standalone

- Jenkins
- Travis
- Circle CI
- TeamCity



Code quality tools

girafe
ai

04

Code styles

- Python standards
  - PEP 8 (code style), PEP 257 (docstrings)
- Custom standards
  - Google Python Style Guide
  - Black code style (rationale)

A Foolish Consistency is the Hobgoblin of Little Minds

One of Guido's key insights is that code is read much more often than it is written. The guidelines provided here are intended to improve the readability of code and make it consistent across the wide spectrum of Python code. As PEP 20 says, "Readability counts".

A style guide is about consistency. Consistency with this style guide is important. Consistency within a project is more important. Consistency within one module or function is the most important.

However, know when to be inconsistent – sometimes style guide recommendations just aren't applicable. When in doubt, use your best judgment. Look at other examples and decide what looks best. And don't hesitate to ask!

Code formatters

Black

The uncompromising code formatter

PyCon talk

Other solutions:

- yapf - configurable
- autopep8 - only pep8
- ruff - fast rust implementation


```
class Foo  (     object  ):
  def f    (self
  ):
    return       37*-2
  def g(self, x,y=42):
      return y
def f  (   a: List[ int ]) :
  return      37-a[42-u :  y**3]
```

```
class Foo(object):
    def f(self):
        return 37 * -2

    def g(self, x, y=42):
        return y


def f(a: List[int]):
    return 37 - a[42 - u : y ** 3]
```

Imports formatter

- isort

  Configurable

  Sorts import to 3 categories:

  - standard library
  - external (third party)
  - internal (this project and associatives)


```
import os
from math import ceil
import myproject.test

import .utils
from math import sqrt
import django.settings
```

```
from math import ceil, sqrt
import os

import django.settings

import myproject.test
```

Linters

- flake8

  List of rules explained
- pylint

Example of rules:

- F401: Module imported but unused
- E402: Module level import not at top of file
- F811: Redefinition of unused name from line n

More solutions from Python Code Quality Authority:

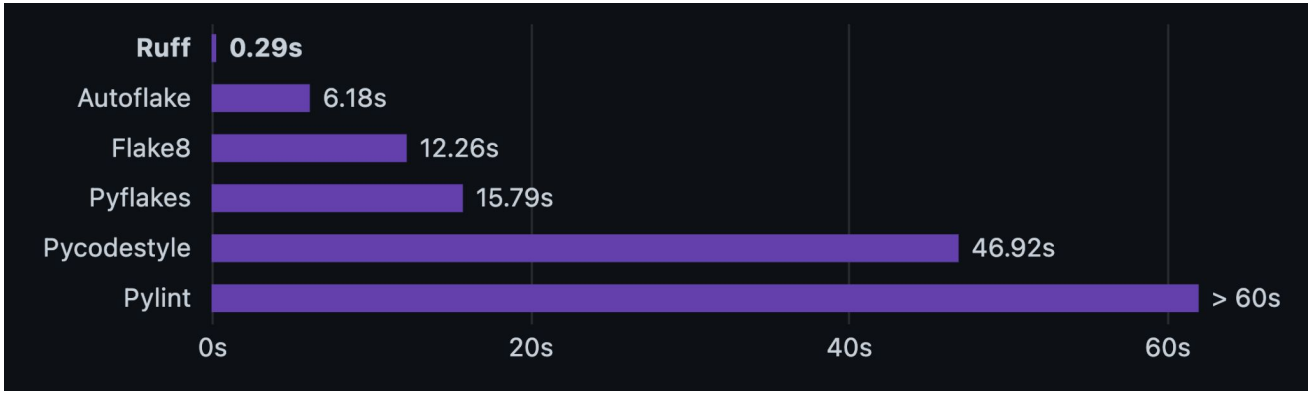
- bandit

Python code quality on rust

- Ruff

More Rust stuff:

- https://github.com/astral-sh/rye
- https://github.com/astral-sh/uv



Typing for Python

- mypy

  Best for code editors,

  not pre-commit

Seamless dynamic and static typing

From Python...

```
def fib(n):
    a, b = 0, 1
    while a < n:
        yield a
        a, b = b, a+b
```

...to statically typed Python

```
def fib(n: int) -> Iterator[int]:
    a, b = 0, 1
    while a < n:
        yield a
        a, b = b, a+b
```

Code quality for Jupyter notebooks

- nbQA

  Same functionality for Jupyter Notebooks!

  Supports pre-commit

Code style for other languages

- prettier
  - YAML
  - TOML
  - Markdown
  - Dockerfile
  - bash
  - etc

Prettier

Pre-commit

pre-commit

uses git hooks to run common utilities before git commands processing

not only for Python and not only for code quality!

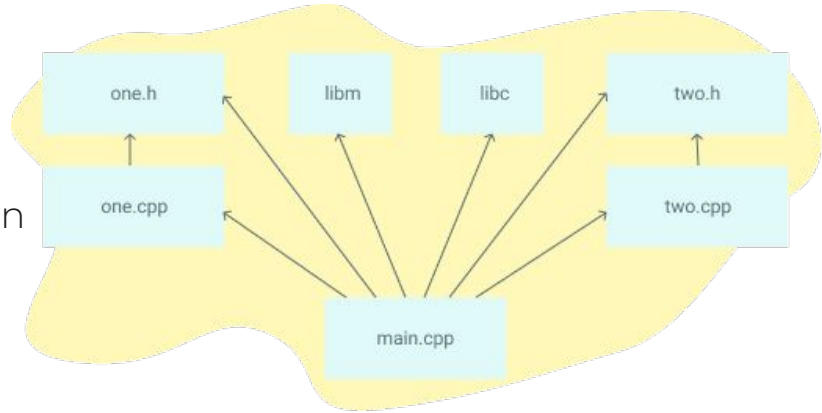

```
$ pre-commit install
pre-commit installed at /home/asottile/workspace/pytest/.git/hooks/pre-commit
$ git commit -m "Add super awesome feature"
black....................................................................Passed
blacken-docs.....................................(no files to check)Skipped
Trim Trailing Whitespace.................................................Passed
Fix End of Files.........................................................Passed
Check Yaml.......................................(no files to check)Skipped
Debug Statements (Python)................................................Passed
Flake8...................................................................Passed
Reorder python imports...................................................Passed
pyupgrade................................................................Passed
rst ``code`` is two backticks.....................(no files to check)Skipped
rst..............................................(no files to check)Skipped
changelog filenames...............................(no files to check)Skipped
[master 146c6c2c] Add super awesome feature
 1 file changed, 1 insertion(+)
```

Alternative: Makefile

https://makefiletutorial.com/

Makefile in context of code quality:

- main goal of Make is building executables (not code quality tools running)
- pre-commit is made for code quality tools
- pre-commit makes separate envs itself
- pre-commit uses git diffed files, not all
- for pre-commit Windows is a first class citizen



So pre-commit is more suitable to run
code quality pipelines.

Standard library tools

- annotations aka type hints
- typing module
- pathlib

Documentation Driven Development

Whole idea behind something driven development is first make something and only then write code.
Test driven development requires considerable effort on test writing.

But documentation is usually needed for any project, so you can write it first.

Learn More

Documentation generation

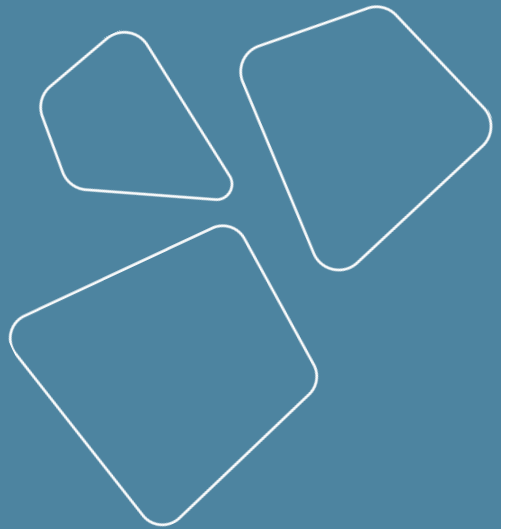
- docstrings
- Sphinx

Tests and coverage

pytest and pytest-cov

https://sourcery.ai/blog/python-best-practices/

Revise



- Хранение кода
- Git servers
- Разработка в больших командах
  - Дистрибуция кода
- CI/CD
- Форматирование кода
- Inline документация

Practice:

- pre-commit
- инструметы контроля качества

Спасибо за внимание!

Вопросы?

girafe ai